Start coding or generate with AI.

LABSHEET 6

NAME: LABSHEET 6

NAME: Saurabh

CU240251626

ROLL NO: 53

```
#1. Build a threshold-based segmentation tool with adjustable global and ada
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg")

if img is None:
    print("Image not found! Check the file path.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, global_threshold = cv2.threshold(
        gray, 127, 255, cv2.THRESH_BINARY
    )

    adaptive_threshold = cv2.adaptiveThreshold(
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, 11, 2
    )

    print("Original Image")
    cv2_imshow(img)

    print("Global Threshold Segmentation")
    cv2_imshow(global_threshold)

    print("Adaptive Threshold Segmentation")
    cv2_imshow(adaptive_threshold)
```

Original Image



Global Threshold Segmentation





Adaptive Threshold Segmentation



```
#2. Develop a region-growing segmentation tool that segments an image starti
import cv2
import numpy as np
from collections import deque
from google.colab.patches import cv2_imshow

img = cv2.imread("/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg")

if img is None:
    print("Image not found! Check the file path.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    x, y = 100, 100
    tolerance = 15

    seed_value = int(gray[y, x])

    height, width = gray.shape
    mask = np.zeros((height, width), np.uint8)
    visited = np.zeros((height, width), bool)

    queue = deque([(x, y)])
    visited[y, x] = True

    while queue:
        cx, cy = queue.popleft()

        if abs(int(gray[cy, cx]) - seed_value) <= tolerance:
            mask[cy, cx] = 255

            for nx, ny in [
                (cx + 1, cy), (cx - 1, cy),
                (cx, cy + 1), (cx, cy - 1)
            ]:
                if 0 <= nx < width and 0 <= ny < height:
                    if not visited[ny, nx]:
                        visited[ny, nx] = True
                        queue.append((nx, ny))

    print("Original Image")
    cv2_imshow(img)

    print("Region Growing Segmentation")
    cv2_imshow(mask)
```

Original Image



```
#3. Create a split-and-merge segmentation demo on a textured or satellite im
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread("/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg")

if img is None:
    print("Image not found! Check the file path.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    output = img.copy()

    def split(x, y, w, h, min_size=24, var_limit=180):
        roi = gray[y:y+h, x:x+w]
```

```
        if roi.size == 0:
            return

        if w <= min_size or h <= min_size or np.var(roi) <= var_limit:
            cv2.rectangle(
                output,
                (x, y),
                (x+w-1, y+h-1),
                (0, 255, 0),
                1
            )
            return

        w2, h2 = w // 2, h // 2

        if w2 == 0 or h2 == 0:
            return

        split(x, y, w2, h2, min_size, var_limit)
        split(x+w2, y, w-w2, h2, min_size, var_limit)
        split(x, y+h2, w2, h-h2, min_size, var_limit)
        split(x+w2, y+h2, w-w2, h-h2, min_size, var_limit)

    split(0, 0, gray.shape[1], gray.shape[0])

    print("Original Image")
    cv2_imshow(img)

    print("Split-and-Merge Segmentation")
    cv2_imshow(output)
```



Original Image



```
#4. Build a watershed segmentation tool to separate touching or overlapping
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread('/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg')

if img is None:
    print("Error: Image not found. Check the file path.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(
        gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )
```

```
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(
    thresh, cv2.MORPH_OPEN, kernel, iterations=2
)

sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist_transform = cv2.distanceTransform(
    opening, cv2.DIST_L2, 5
)

_, sure_fg = cv2.threshold(
    dist_transform,
    0.4 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

markers = cv2.watershed(img, markers)

result = img.copy()
result[markers == -1] = [0, 0, 255]

print("Original Image:")
cv2_imshow(img)

print("Watershed Segmentation:")
cv2_imshow(result)
```



Original Image:



```
#5. Develop a K-means colour segmentation tool.
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread('/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg')

if img is None:
    print("Error: Image not found. Check the file path.")
else:
    pixel_values = img.reshape((-1, 3))
    pixel_values = np.float32(pixel_values)

    criteria = (
        cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
```

```
    100,
    0.2
)

k = 3

_, labels, centers = cv2.kmeans(
    pixel_values,
    k,
    None,
    criteria,
    10,
    cv2.KMEANS_RANDOM_CENTERS
)

centers = np.uint8(centers)
segmented = centers[labels.flatten()]
segmented = segmented.reshape(img.shape)

print("Original Image:")
cv2_imshow(img)

print("K-Means Segmented Image:")
cv2_imshow(segmented)
```



Original Image:



```
#6. Create an interactive foreground-extraction tool using GrabCut.
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread('/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg')

if img is None:
    print("Error: Image not found. Check the file path.")
else:
    mask = np.zeros(img.shape[:2], np.uint8)

    height, width = img.shape[:2]
    rect = (10, 10, width - 20, height - 20)
```

```
bgd_model = np.zeros((1, 65), np.float64)
fgd_model = np.zeros((1, 65), np.float64)

cv2.grabCut(
    img, mask, rect,
    bgd_model, fgd_model,
    5, cv2.GC_INIT_WITH_RECT
)

mask2 = np.where(
    (mask == 2) | (mask == 0), 0, 1
).astype('uint8')

foreground = img * mask2[:, :, np.newaxis]

print("Original Image:")
cv2_imshow(img)

print("Extracted Foreground:")
cv2_imshow(foreground)
```



Original Image:



```
#7. Build a normalized-cut-based segmentation demo.
import cv2
import numpy as np
from sklearn.cluster import SpectralClustering
from google.colab.patches import cv2_imshow

img = cv2.imread('/content/WhatsApp Image 2026-08-29 at 12.17.11 AM.jpeg')

if img is None:
    print("Error: Image not found. Check the file path.")
else:
    small = cv2.resize(img, (100, 100))

    pixels = small.reshape((-1, 3)).astype(np.float32) / 255.0

    clustering = SpectralClustering(
```